THE UNIVERSITY OF WARWICK

Paper Details

Paper Code: CS2580

Paper Title: Database Systems

Exam Period: April 2025

Exam Rubric

Time Allowed: 2 hours

Exam Type: Written - Standard

Approved Calculators: Permitted

Instructions

Answer **FOUR** questions.

If more than **FOUR** questions are answered, please clearly indicate which questions you wish to be marked. Otherwise, the first **FOUR** questions answered in the answer booklet will be marked.

Read carefully the instructions on the answer booklet and make sure that the particulars required are entered on each answer booklet.

The exam question paper **MUST NOT** be removed from the examination venue.

Permitted calculators: CASIO FX82, FX83 and FX85 series.

WARWICK
THE UNIVERSITY OF WARWICK

Question 1 [25 marks]

The following schema represents a university course management system, where each course (a.k.a module) is divided into a number of sessions every year, offered at different times):

- Department (dept_id, dept_name, building, budget)
- Professor (prof_id, name, dept_id, hire_date, salary)
- Course (course_id, title, dept_id, credits)
- Session (session_id, course_id, term, year, professor_id, room, time_slot)
- Student (student_id, name, dept_id, credits_earned, overallmark)
- Enrollment (student_id, session_id, grade)

a) Write a SQL query to find all students who are taking courses in their department in term 2 of 2025. Display student names and course titles. Each student should only be displayed once.
You must use explicit join operations (… JOIN … ON…). You must not use subqueries.
[5]

b) Write a SQL query to calculate the average final mark of students in each department but only consider departments that have at least 10 students. Order results by average overallmark in descending order. Show department name and average overallmark.
You must use explicit join operations (… JOIN … ON…). You must not use subqueries.
[5]

c) The university's policy requires that no session can have more than 50 students. Write a SQL query to identify any sections that violate this policy. The query should return the course title, session_id, professor name, and the number of enrolled students.
You must use explicit join operations (… JOIN … ON…). You cannot use subqueries.
[6]

d) Write a SQL query to find pairs of professors from the same department whose salaries are within £5,000 of each other. The query should return professor names, department name, and their salaries.

You must use explicit join operations (… JOIN … ON…). You must not use subqueries.
[9]

Question 2 [25 marks]

Consider the following schema, S1.

- Students (student_id, name, enrollment_year)
- Courses (course_id, course_name, department)
- Enrollments (student_id, course_id, grade, term)
- Professors (professor_id, name, department)
- Teaches (professor_id, course_id, term)

a) Consider the following SQL query.

```
SELECT      e.course_id, c.course_name, s.name AS student_name, e.grade
FROM        Enrollments e
JOIN        Students s ON e.student_id = s.student_id
JOIN        Courses c ON e.course_id = c.course_id
WHERE       term = 1 AND e.grade = (
        SELECT MAX(grade)
        FROM Enrollments
        WHERE course_id = e.course_id AND term = 1
);
```

Explain in one sentence what the query does.

[5]

b) Consider the following query.

```
SELECT      s.student_id, s.name
FROM        Students s
WHERE NOT EXISTS (
                SELECT c.course_id
                FROM Courses c
                WHERE c.department = 'Computer Science'
                AND NOT EXISTS (
                        SELECT e.student_id
                        FROM Enrollments e
                        WHERE e.student_id = s.student_id AND e.course_id = c.course_id
                )
);
```

Explain in one sentence what the query does.

[7]

c) Consider the following two queries, Q1 and Q2.

Q1

```
SELECT e.student_id, s.name
FROM Enrollments e
JOIN Students s ON e.student_id = s.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE c.department = 'CS'
GROUP BY e.student_id, s.name
HAVING COUNT(DISTINCT e.course_id) = (
      SELECT COUNT(*)
      FROM Courses
      WHERE department = 'CS'
);
```

Q2

```
SELECT s.student_id, s.name
FROM Students s
WHERE NOT EXISTS (
  SELECT c.course_id
  FROM Courses c
  WHERE c.department = 'CS'
  AND NOT EXISTS (
    SELECT e.course_id
    FROM Enrollments e
    WHERE e.student_id = s.student_id
    AND e.course_id = c.course_id
  )
);
```

Is Q1 equivalent to Q2 (i.e., do they return the same answer)? If your answer is YES, explain in one sentence what the queries are doing. If your answer is NO, explain in one sentence how Q1 may be changed to do what Q2 does.

[13]

---

continued

Question 3 [25 marks]

a)

A bank stores customer details in the following database:

Customer(CustomerID, Name, Address, Salary, CreditRating)

Assume CustomerID uniquely identifies a customer.

Each staff member is allocated a clearance in the range 1 to 4. Users at level k, 1<k<=4, can see data at levels 1 to k.

Explain how a relational database system could be modified to provide different information to different members of staff. Illustrate your answer by explaining:

- How the DBMS selects the appropriate data tailored to the clearance of staff member
- The problems that could arise if a staff member of level 2 tries to add data about a customer who already exists but only has information stored at level 3, and how this problem can be addressed.

[8]

b)

A government agency is conducting a survey to collect data from millions of individuals about sensitive topics such as income levels, job title and postcode, and whether or not the individual has a health condition. The agency aims to use this data to generate reports that inform public policy decisions. Statisticians will analyse the data, but should not be allowed to see data about individuals.

Explain the problems that could occur with this scenario, and explain techniques that will protect respondents' privacy whilst allowing accurate and reliable analysis of the survey results. Explain why it might be useful to add some 'incorrect' data to the dataset, and how this can be accounted for during analysis.

[12]

c)

A researcher has created a web application that interacts with a database to store and retrieve user information. When a user logs in, the application checks whether login details are valid using the below code:

query = "SELECT 1 FROM loginTable WHERE username = '" + username + "' AND password = '" + password + "'";

During testing, the researcher verified the login process was working, but after deploying the system, it was noted that all passwords were now set to an empty string. Explain how this could have happened and explain the best way of eliminating this problem.

[5]

Question 4 [25 marks]

a)
For each of the following databases, with the given functional dependencies, state the key of each relation, and explain the Normal Form of the database (with a brief explanation of why it can't be in a lower and/or higher normal form).

State any assumptions made about the contents of the data.

i) Database 1:
R1(A, B, C), R2(A,D,E,F,G)
A→{B,C}
E→{G}
{A,D}→{E,F,G}

[5]

ii) Database 2:
R1(A, B, C), R2(D,E,F,G)
{A,B}→C
{D,E}→G
D→F

[4]

iii) Database 3:

R1(A, B), R2(A,C,D), R3(A,C,E), R4(A,D, F)
A→B
{A,C}→D
{A,C}→E
{A,D}→F

[4]

b)

Consider the table:
Customers(CustomerID, CustomerName, CustomerAddress)

Given that CustomerID → {CustomerName, CustomerAddress}, state the name of the rule that demonstrates CustomerID → CustomerAddress, and provide a full proof of the rule. You may abbreviate the names of the attributes.

[4]

c)

Given a relation R(A,B,C,D,E,F) and the functional dependencies
A→{B,C}
B→E
{C,D}→{E,F}

Show that {A,D} → F. You may use any of Armstrong's Axioms without proof, but you must provide the names of each rule used.

[5]

d)
Define the two properties required or desired for an appropriate decomposition and state the potential consequences if either property is not enforced in the database design.

[3]

Question 5 [25 marks]

A bookshop stores data in the following relations:

Genre(genreID, genreName)

Book(bookID, title, author, publisher, year, price, genreID)

Set(bookID, SetID, position)

Customer(customerID, customerName, birthyear)

Order(customerID, bookID, purchasedate)

SetDetails(SetID, SetName)

All ID fields (genreID, bookID, SetID, customerID) are integers.

The fields position, birthyear and year are integers.

Price is numeric and represents price in £.

The field purchasedate is an integer value of the number of days since 1st January 2000.

Some books belong to a set (a series of books). For example, book 1 of the "A University Guide to Computing" set might be "Basic Programming," book 2 might be "Intermediate Programming," and book 3 might be "Advanced Programming."

Each position is sequential – for example, it is impossible to have a book 5 without a book 4.

a) Write a relational algebra query for the following. You must use only the core relational algebra operators (you **must not** use aggregation or outer join operators)

i) Retrieve the names of authors who have written a book from the 'Computing' genre.

[3]

ii) Retrieve the titles of all books from sets where a book was published more than 10 years after the previous book in the same set.

[10]

b) Write a Tuple Relational Calculus expression to retrieve the names of customers who have ever spent more than £20 on a book that is not part of a set.

[7]

c) Explain what is returned by this Domain Relational Calculus expression.

```
{i, b | Book(a,b,c,d,e,f,g) AND Customer(h,i,j) AND Order(k,l,m)
AND k = h AND l=a AND Genre(n,o) AND n = g AND Set(p,q,r) AND p
= a AND (r = 1 AND j > 1990 AND o = 'Computing' AND f > 40) OR
f < 20}
```

[5]

End